Tiny Scale Is All I Can Spare To Play With Transformer.

Srijan Srivastava
India
Srivastavavsrijan321@gmail.com
QCoreNest@gmail.com

Abstract

Introduction of the Transformer neural network architecture in the famous `Attention Is All You Need` paper has created a huge wave of AI development in recent years. The scaled dot-product attention allows for information to be processed with higher efficiency and quality, which the previous RNN-based models lacked. However Transformer-based models comes with their own challenges, particularly with parameter efficiency for tiny models with parameters ≤ 5M. At such small scale a Transformer model essentially uses more parameter than it really should. This sub-ten-million parameters domain space is very underexplored and for good reasons but I wanted to explore it anyways. So here-in this paper I am introducing Silia, a novel transformer architecture designed for efficient modelling & classification tasks under severe parameter budget. Training against GPT-2 architecture (Andrej Karpathy's nanoGPT project) with same "base" hyperparameters, training data and compute budget, Silia achieves comparable loss and generation quality with significantly less parameters.

> **Note**
>
> Due to the following constraints I'll be treating this paper more as a report on the architecture than a final conclusion of any means.
>
> All experiments were conducted on an old (2nd Gen Intel Core i3 processor, 8 GB RAM and no dedicated GPU) hardware reflecting resource reality of many independent student researchers, including me.
>
> This also means that Silia's parameter efficiency and performance gains may fade away at larger scales. Though this is still to be explored since I haven't scaled up Silia beyond 10M parameters.

1. Introduction

The dominant trend in Transformer-based language models has been scaling: larger models, more data, more compute in pretraining and reinforcement learning for chain-of-thought based reasoning capabilities to consistently yield better performance. This means that the smallest practical models lie anywhere between 1B-20B parameters, such as Qwen,

Gemma & GPT-OSS. Despite them being called "small", these models are still billions in parameters and are trained on trillions of tokens. This trajectory, as useful as it is, has widened the gap between general-purpose frontier research experiments and task-specific small research experiments.

So why not just scale down standard Transformer models? The reason is that Transformer is a very parameter heavy architecture and as much those parameters help at larger scales, most of those parameters are just wasted at tiny scale such as sub-ten-million (≤ 5M) parameters. Now there are several ways to make Transformers parameter efficient, such as tied embedding weights, factorized linear layers, smaller expansion in feed-forward networks and more, but these aren't enough.

This is where the Silia neural network architecture enters the chat. Silia merges the `Attention` layer with the `FFN` layer to save lots of parameters while preserving much of the original performance. This new Attention-merged-FFN layer is what I call `Silia` or Silu in Attention as I took SwiGLU as the base FFN layer.

2. Silia (Silu In Attention)

Attention and SwiGLU layer are both merged into a single layer called **Silia**

2.1. The Mathematics

2.1.1. SwiGLU

SwiGLU was introduced and used by Google in their (N Shazeer, 2020) paper `GLU Variants Improve Transformer`.

$$\text{SwiGLU}(X) = (\text{SiLU}(XW_1) \odot XW_2)W_3$$

$$\text{SiLU}(XW_1) = XW_1 \cdot \sigma(XW_1) = \frac{XW_1}{1 + e^{-XW_1}}$$

Here, X is the hidden state of our input from previous layer. W_1 & W_2 are linear matrices. W_3 is the output projection matrix. $\odot$ is the element-wise multiplication operation. σ is the sigmoid activation function.

X usually has a shape `(B, T, C)`. W_1 & W_2 has a shape `(C, 4*C)` each and W_3 has a shape `(4*C, C)`. Where `B` is batch size, `T` is sequence length and `C` is the embedding dimension.

Prior to this, the standard Transformer architecture heavily relied on simpler activations like ReLU or GeLU. The paper demonstrated that replacing standard feed-forward network layers with Gated Linear Units (GLUs), specifically those utilizing the *Swish* activation function (SwiGLU) significantly improved training convergence and downstream model accuracy. Since then SwiGLU has been a popular choice for researchers to use in their Transformer models.

2.1.2. Attention

Attention is the heart of Transformer and it needs no introduction, so before diving into Silia's mathematics let's get attention's done first.

$$Q = XW_Q, K = XW_K, V = XW_V$$

$$\text{MHA}(Q, K, V, M) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

$$\text{Attention}(X) = \text{MHA}(Q, K, V, M)W_O$$

The above equation is what was introduced in the now famous `Attention Is All You Need` paper. This is the equation which is used for autoregressive language modelling where W_Q is the query matrix, W_K is key matrix, W_V is value matrix, W_O is output projection matrix, M is a causal attention mask and d_k is dimension of the key vectors.

2.1.3. Silia

Now as we've been through both SwiGLU and Attention, let's get into the mathematics of **Silia**.
We first calculate attention over the hidden state X.

$$Q = XW_{Q_1}, K = XW_{K_1}, V_1 = XW_{V_1}, V_2 = XW_{V_2}$$

$$\text{AttentionScores}(Q, K, V, M) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)$$

$$U = AttentionScores(Q, K, V, M)V_1$$

$$V = AttentionScores(Q, K, V, M)V_2$$

$$H = \text{SiLU}(U) \odot V \tag{1}$$

Now in equation (1) we have applied the $SiLU$ activation function on the linear transformation of our hidden state X via attention mechanism. Now we will calculate attention over this new hidden state H.

$$Q = HW_{Q_2}, K = HW_{K_2}, V = HW_{V_3}$$

$$\text{MHA}(Q, K, V, M) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

$$O = \text{MHA}(Q, K, V, M)W_O$$

And here we go. We have our new **Silia** feed-forward network.

2.2. How Is It Parameter Efficient?

Now at first glance it looks like we are increasing parameters per layer rather than decreasing but it's actually quite opposite. Let me clarify (I will be excluding all biases for easier calculations).

2.2.1. Parameters Per Layer In Transformer

In a traditional Transformer the Attention layer has W_Q, W_K, W_V and W_O matrices, and SwiGLU has W_1, W_2 and W_O matrices. For the sake of simplicity let's ignore the W and call it all QKV (Query, Key, Value) in attention and GC (Gate, Content) in SwiGLU.

1. Matrix QKV is shaped `(C, 3*N*D)` where Q, K, V each `(C, N*D)`.
2. Matrix O in Attention is shaped `(N*D, C)`.
3. Matrix GC is shaped `(C, 2*4*C)` where G, C each `(C, 4*C)`.
4. Matrix O in SwiGLU is shaped `(4*C, C)`.

Here `C` is embedding dimension, `D` is head dimension (typically 64 or 128), `N` is number of heads, `4*C` in SwiGLU is expansion. Head dimension (`D`) is typically `C/N`.

Now add all Attention layer shapes

$$C * (3 * N * D) + (N * D) * C = 4 * C^2 \tag{1}$$

Now add all SwiGLU layer shapes

$$C * (2 * 4 * C) + (4 * C) * C = 3 * 4 * C^2 \tag{2}$$

Now add equation (1) and (2) together

$$4 * C^2 + 3 * 4 * C^2 = (4 * C)^2$$

So we have a total of $(4 * C)^2$ parameters per layer in a traditional Transformer.

2.2.2. Parameters Per Layer In Silia

Unlike Transformer in Silia we merge both Attention and SwiGLU FFN together as discussed above. In Silia we have W_{Q_1}, W_{Q_2}, W_{K_1}, W_{K_2}, W_{V_1}, W_{V_2}, W_{V_3}, W_O layers. Again for the sake of simplicity, let's call them QK, V and O.

1. Matrix QK is shaped `(D, 4*N*D)` where W_{Q_1}, W_{Q_2}, W_{K_1} and W_{K_2} each is shaped `(D, N*D)`.
2. Matrix V is shaped `(D, 3*N*D)` where W_{V_1}, W_{V_2} and W_{V_3} each is shaped `(D, N*D)`.
3. Matrix O is shaped `(N*D, D)` -> `(N*D, D)`.

Here `D` is head dimension (set to 64 or 128), `N` is number of heads.

Now add all shapes together

$$D * (4 * N * D) + D * (3 * N * D) + (N * D) * D = 8 * N * D^2$$

So in Silia we have $8 * N * D^2$ parameters per layer. Comparing number of parameters in Transformer with Silia we get $8 * N * D^2 < (4 * C)^2$

2.2.3. What Just Happened?

Now I know what you might be thinking. *Why did he take embedding dimension same as head dimension in his architecture? Obviously this will reduce the number of parameters. Is he stupid? Did I just wasted my time reading some AI slop?*

No. Let me explain.

Since all our linear computation is now happening within the attention mechanism, we don't need traditional expansion bottlenecks anymore. This means that keeping the embedding dimension similar to head dimension of traditional Transformer (between 64-512) is more than enough for our model actually. Expansion does happen but instead of expanding on embedding dimension, we expand on the number of heads.

Especially considering we are designing Silia to work with tiny scale this approach makes more sense and also helps us save quite a lot of parameters. Let's compare Silia with traditional Transformer.

Hyperparameters	**Transformer**	**Silia**
embd dim (`C`)	256	64
heads dim (`D`)	64	64
num heads (`N`)	4	4
num params per layer	1,048,576 (1.04M)	131072 (0.13M)

So with Silia we saved 87.5% parameters from the Transformer.

Hyperparameters	**Transformer**	**Silia**
embd dim (`C`)	1024	64
heads dim (`D`)	64	64
num heads (`N`)	16	16
num params per layer	16,777,216 (16.77M)	524,288 (0.52M)

So with Silia we saved 97% parameters from the Transformer.

Please note that in Silia embedding dimension (`C`) and head dimension (`D`) are exactly the same thing.

2.3. The Intuition

So how I ended up with this idea? Why do I think replacing linear layers in SwiGLU with Attention layers was a good idea?

The reason is that we already know that Attention is mostly a linear transformation over our hidden state, but the catch is that it's a "smarter" linear transformation.

We can observe in standard Transformer, attention layer modifies the hidden state then FFN processes that "smartly" modified hidden state. However an attention-only Transformer perform incredibly poorly (not as bad a linear-layer-only-no-non-linearity neural network) due to the lack of a "strong" activation function. Attention only has `softmax` as it's activation function but `softmax` only decides which tokens should attend to which other tokens. This makes `softmax` a not so "strong" activation function.

Attention is dynamic and smart about which information to mix, but it has no strong non-linearity to actually transform that information. SwiGLU has the strong non-linearity but it's static. Same weights for every input.

As per my observation SwiGLU at small scales (less parameters and specially context windows) can approximate what attention does (token mixing and information routing). However it fails at longer contexts because it has no positional awareness or dynamic weighting.

So instead of running both separately and wasting parameters on overlapping functionality, Silia replaces the static linear matrices in SwiGLU with attention getting dynamic mixing and strong non-linearity in one unified operation.

3. Experiments

The idea and intuition is quite simple but it works surprisingly well at tiny scale (≤ 5M parameters) and is able to achieve comparable loss and generation quality to Andrej Karpathy's GPT-2 architecture based nanoGPT model.

The following experiments were conducted on an old hardware with 2nd Gen Intel Core i3 processor, 8 GB RAM and no dedicated GPU so there isn't much experimental evidence to back any of my claims but they do show interesting results and up to some degree agree with my hypothesis.

I custom trained a OpenAI-o200k_base-regex-pattern+BPE tokenizer on my custom Srijan-Srivastava/super-tiny-webtext dataset with a vocabulary size of 8192 tokens. This exact same tokenizer was used for all the following experiments.

Hyperparameters for Silia:

1. Block size = 256 (with rotary embedding can scale up to 2k)
2. Number of layers = 2
3. Number of heads = 4
4. Embedding size = 64
5. Batch size = 16
6. Max iterations = 10,000
7. Max learning rate = 3e-3
8. Min learning rate = 3e-4

The model had 0.786432M total parameters, out of which 0.262144M were non-embedding parameters.

Hyperparameters for nanoGPT (GPT-2 with RoPE & SwiGLU):

1. Block size = 256 (with rotary embedding can scale up to 2k)
2. Number of layers = 2
3. Number of heads = 4
4. Head dimension = 64
5. Embedding size = 256
6. Batch size = 16
7. Max iterations = 10,000
8. Max learning rate = 3e-3
9. Min learning rate = 3e-4

The model had 4.19M total parameters, out of which 2.09M were non-embedding parameters.

> **Note**
>
> I'll be only attaching the generated sample outputs from Silia and not from nanoGPT because the outputs are very similar.

3.1. Business Email Generation

Inspired from Experiment: How far can a 28M model go in business email generation? I trained a business email generation model on Kamisori-daijin/email-datasets-20k on Hugging Face. There are 20,000 samples of emails and was created using **Gemma 3-4B-it** (via mlx-community/gemma-3-4b-it-4bit-DWQ).

Post-tokenization the dataset had 5.51M tokens, with 80/20 rule I divided it into 4.40M training tokens and 1.10M validation tokens. Both models were trained on 8.2 epochs.

After training the final losses looked like this:

Loss	Silia	nanoGPT
Training	1.43	1.4
Validation	1.48	1.44

Here's one sample output:
Prompt: " `Write a polite refusal email<|eop|>` **"**

From the ~28M parameter model

```
I understand this is a Friday evening, but I'm happy to provide more
information.
I'll do my best to discuss the details and explore possible alternatives.

We'll keep you updated on our progress. Please let me know if this is
something you'd be interested in.

Best,
[name]
```

From my model

```
Correcting Reason for Recent Experience
Dear [Officer's Name],

I hope this email finds you well.

As you know, [Company Name] has recently received your experience with
[Product Name], a hugely successful funding round, and we're committed to
providing you with the billing records and targeted a highly value of
[Client Name].

As you know, our rebranding initiative is currently undergoing a significant
rebranding phase, and we want to sincerely apologize for the timing. We
value your time and appreciate your understanding.

We understand your concerns swiftly and want to assure you that we will be
moving forward. Please let us know your availability.

Sincerely,
[Name]
Junior Developer
[Company Name]<|eot|>
```

It is still inconsistent but occasionally gets close.

3.2. WebText Generation

Trained on my custom Srijan-Srivastava/super-tiny-webtext dataset on Hugging Face. There are 1447 samples and was created by scrapping and cleaning very specific webpages on various topics.

This dataset contain texts from Wikipedia (on various topics, personalities, games, movies, companies and more), fandoms, storylines, scrips and story dialogues of various games (such as GTA, RDR, Last of Us, Mafia, Cyberpunk 2077 and more), transcripts of some YouTube videos, several research papers, academic articles and blogs (mainly revolving around AI and LLMs in general) and code from some of my personal code bases and other

public repositories such as the Hazel Game Engine repository on GitHub. I tried my best to keep the programming languages limited to just Python, C#, C++ and JavaScript in the dataset. All of this made ~30M characters in total.

Post-tokenization the dataset had ~9M tokens, with 80/20 rule I divided it into ~7M training tokens and ~2M validation tokens. Both models were trained on 5.8 epochs.

After training the final losses looked like this:

Loss	Silia	nanoGPT
Training	3.46	3.15
Validation	3.85	3.44

Here's one sample output:
Prompt: " `Steve Jobs made the soul of Apple` **"**

```
Steve Jobs made the soul of Apple and he had a run with Pixar on the company
later in a decade I was transported in June 2011, but h e made Jobs was a
school case that the company had been accessible to his chairman to fill me
named Steve Jobs and Jobs, in the com pany and encounters a $10.5 million in
2022 demanded Apple studio to Apple. In 2023, Jobs announced OpenAI's
announcement of the fil m, "Skaxicly that he looks changing the company of
the musician, the Macintosh coding virtual businesses but he stated that he
wasbl ished for their share of other company. Thing the first Pixar Low and
film received a greater of Marvel Studios in US$1.5 million so ld to the
board of directors in the late 2011, and the previous release was officially
based on funding for the film for launching t he most of the Year. The
company's "Didau 16.5 million for the effort to make it a company to frames
the company of the Solution, an d having a more access to the company of
Apple II. [216] The Wall, Thain,
```

3.3. ChatAlpaca Generation

Trained on ChatAlpaca: A Multi-Turn Dialogue Corpus based on Alpaca Instructions dataset on Hugging Face. There are 20,000 samples and was created using **GPT-3.5-turbo** to generate follow-up utterances and continue the conversation with ChatGPT. This process results in multi-turn conversations where the simulated user provides instructions and ChatGPT responds accordingly.

Post-tokenization the dataset had ~18M tokens, with 80/20 rule I divided it into ~14.4M training tokens and ~3.6M validation tokens. Both models were trained on 2.8 epochs.

After training the final losses looked like this:

Loss	Silia	nanoGPT
Training	3.35	2.98
Validation	3.4	3.41

Here's one sample output:
Prompt: " `Describe the process of decision tree learning.<|eop|>` **"**

```
Describe the process of decision tree learning.<|eop|>
Decision tree learning is a supervised machine learning model that uses
language, powers, and other machine learning can be used to perform datasets
that handle no parsion. Additionally, it would include the test and
visualization form of an AI language model. an
AI-powered speech recognition technology that indicate sound quality and
meaning. In this sentence, the neural networks, computers c an be used to
identify whether the text, while both being used in text. It is more
relevant, accurate and more accurate than text an imations or conversational
animals and animals. G<|eot|>
```

4. Conclusion

4.1. Use Cases

1. It can be used as super-light-weight, attention-powered, on-device models in Smart Watches, old Mobile Phones and several generations old computers for very task-specific generations and classification.
2. It can be used as on-device models to immediately generate dialogues for NPCs in video games increasing immersion.
3. It can be used for simple & fast image/text/topic classification, sentiment/emotion analysis, intent/toxicity detection and more.

4.2. Limitations

I was unable to test this architecture beyond 5M parameters due to my hardware limitations. This is something which I want to focus on getting done next. I did try using Google Collab but since I do not have a subscription I wasn't able to scale the number of parameters a lot. Though using Google Collab meant that the training finished within a few minutes rather than hours on my PC.

I believe that at larger scales (beyond 50M parameters) Silia will break due to it's extremist focus on parameter efficiency. By "break" I mean that since Silia is using Attention in place of linear transformation matrices we cannot increase the embedding/head dimension beyond 512 or increase the number of heads a lot as there are already certain observations made that Attention with 32 heads and 64 head dimension performs way better than with 4 heads and 512 head dimension.

The only practical way to scale Silia is by adding more layers rather than increasing head dimension or number of heads, since Silia is already constrained in both. This means at larger scales Silia would require significantly more layers than an equivalent Transformer, making it slower to train and run.

4.3. Closing Thoughts

Silia is a small idea for a small scale. The sub-10M parameter space is underexplored and for good reason, there isn't much glory in it. But I think there's some genuine value in asking whether the standard Transformer block is the right design when you only have a few hundred thousand parameters to spare. Merging attention and SwiGLU into a single unified operation isn't a revolutionary idea, but the parameter savings are real and the results are encouraging enough to be worth sharing. I hope this paper is useful to someone working in the same constrained corner of the field that I am.

References

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, (2017). Attention is all you need. *arXiv preprint arXiv:1706.03762*.

Noam Shazeer, (2020). GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*.

Andrej Karpathy, (2022). nanoGPT. GitHub. https://github.com/karpathy/nanogpt